Asteroids: Data and Architecture Design

Ahmet Sevinç: 6962084
Bram den otter: 7153651
6 November, 2026

*updated

1 Introduction and rules

This document describes the implementation we have planned for our Asteroids game. Asteroids is a singleplayer game in which the player controls a ship in space. The goal of the game is to score as many points as possible, by shooting down moving asteroids and ufo's.

The ufo shoots bullets to try to kill the player. The asteroids are split when hit unless they are the smallest size then they disappear. The asteroids move in a random direction and are the same as the player wrapped in the screen.

We are not going to implement a real twist to the game.

We have tried to maintain a clean code style and commented where necessary.

2 Design

2.1 Data Structures

2.1.1 The Game State

```
data GameState = GameState
  { player      :: Ship
  , bullets     :: [Bullet]
  , asteroids   :: [Asteroid]
  , ufo         :: [Ufo]
  , score       :: Score
  , highScore   :: Score
  , lives       :: Lives
  , elapsedTime :: Float
  , paused      :: Bool
  , gameStatus  :: GameStatus
  , deathTimer  :: Float
  , invincibilityTimer :: Float
  , controls    :: Controls
```

```
  }
  deriving (Eq, Show)

data GameStatus = Playing | Dying | GameOver
 deriving (Eq, Show)


Type Score = Int
Type Lives = Int
```

2.1.2 Ship, Asteroids, Ufo’s and Bullets

```
data Ship = Ship
  { shipPosition :: Position
  , shipRotation :: Rotation
  , shipVelocity :: Position
  , shipSize     :: Float
  }
  deriving (Eq, Show)

data Asteroid = Asteroid
  { asteroidPosition :: Position
  , asteroidVelocity :: Position
  , asteroidSize     :: AsteroidSize
  }
  deriving (Eq, Show)

data AsteroidSize
  = Large
  | Medium
  | Small
  deriving (Eq, Show)

data Ufo = Ufo
  { ufoPosition :: Position
  , ufoVelocity :: Position
  , ufoSize     :: Float
  }
  deriving (Eq, Show)

data Bullet = Bullet
  { bulletPosition :: Position
  , bulletVelocity :: Position
  , bulletSize     :: Float
  , bulletTime     :: Float
  }
  deriving (Eq, Show)
```

```
data Position = Position
  { x :: Float
  , y :: Float
  }
  deriving (Eq, Show)

Type Row = Int
Type Column = Int
```

2.1.3 Player Movement

The player controls the ship, which moves smoothly through space in all possible directions. The ship always stays in the screen, because he is wrapped inside it. If the ship leaves the screen on one side it will come out the opposite side. The ship also has friction so it will not move forever with 1 input.

```
accelerateShip :: Float -> Ship -> Ship
updateShip :: Float -> Ship -> Ship
rotateShip :: Float -> Ship -> Ship
wrapPosition :: Position -> Position
```

The updateShip function updates the player's position based on the current direction, and wraps if necessary.

2.2 Computer Adversary

In the game, the Asteroids will be acting as one of the adversaries. Simply by the fact that they can destroy the player’s ship by colliding with them.

Another adversary will be the UFO. The UFO keeps track of the position of the player, and will be moving towards them. This will be its “intelligent” behaviour.

2.3 Interface

The game is rendered using the Gloss library, with the screen displaying the ship, asteroids, ufo’s and bullets. These entities are drawn using geometric shapes and their current position. The ship rotates and moves according to the inputs of the player, while the asteroids and ufo’s move continuously.Bullets are shown as small circles. The interface shows the current score, highscore, remaining lives and the time. There is also a paused and gameover screen.

Input is processed via Gloss events, such as arrow keys for direction and p for pause.

2.4 Implementation of the Minimum Requirements

- **Player**. The player controls the ship, using the keyboard. Keyboard input is captured by an input processing function that updates the player's desired direction. The movement logic reads this desired direction and moves

  We have implemented data Ship.
- **Enemies**. The enemies are asteroids and ufo's, each have their own behaviour. Asteroids move continuously in a random direction.

ufos actively pursue the ship of the player by moving towards its current position.
We have implemented data Ufo and Asteroid
- **Randomness**. In the beginning of every game the Asteroids and the Ufo are initialized to a random position on the screen, but not too close to the player. Asteroids also have a random size and begin direction.
- **Animation**. Whenever the player collides with any of the adversaries and thus lose a life, an animation will play. In addition to that, the player will be given invincibility frames when they spawn back in. These invincibility frames will also be animated by making the player blink
- **Interaction with the file system.** The program reads and writes the score data in one file.
  We have implemented in data GameState highscore :: Score.

2.5 Implementation of the optional Requirements

We have not implemented optional requirements.

2.6 Splitting pure and impure functions

In our game, we deliberately separated pure and impure code where possible. The pure code is mainly found in game logic, such as movement and collisions.

The impure code is mostly done in HighScore and in the wrapper function of Controller.

The controller handles input events, generates random values for the new asteroids and ufo's and does the IO updates. the highscore module handles writing and reading a file, so is also impure.

So we spread it as much as possible.

2.7 Abstractions

In our game, we use abstraction to avoid code duplication.
One example would be the use of higher-order functions like map. This is useful for several game objects, like asteroids, UFOs and bullets.

We also reuse functions for behaviour shared by different objects. One example for this is

wrapPosition, which would apply to all ships, asteroids and UFOs.
Another one is collision detection, which is used to check for all objects with collision in mind